

INFORME TÉCNICO

Simulación de Intercepción de Projectiles mediante Métodos Numéricos

Asignatura:

Métodos Numéricos / Computación Científica

Autor(es):

Juan Castañeda

Cristhian Espin

Ariel Burgos

Steven Moreta

Fecha:

Mayo 2026

Documento académico — Computación Científica

1. Dependencias y Manual de Instalación y Uso

Listado de Dependencias (`requirements.txt`)

Para garantizar la correcta ejecución del programa y de las simulaciones dinámicas, se requiere un entorno de Python 3.8+ con las siguientes librerías especializadas instaladas:

`numpy>=1.24.0`

`matplotlib>=3.7.0`

`scipy>=1.10.0`

Nota sobre la interfaz gráfica: La librería `tkinter` forma parte de la suite estándar de Python en la mayoría de las distribuciones. Sin embargo, en ciertos sistemas operativos basados en Linux (como Ubuntu o Linux Mint), es necesario instalarla explícitamente a través del gestor de paquetes del sistema operativo mediante el comando `sudo apt-get install python3-tk`.

Manual de Instalación Paso a Paso

1. **Creación de un entorno virtual (Recomendado):** Para evitar conflictos de dependencias con otros proyectos de Ciencia de Datos o Software, aisle el entorno ejecutando en su terminal:

```
python3 -m venv env_metodos  
source env_metodos/bin/activate # En Linux/macOS  
env_metodos\Scripts\activate # En Windows
```
2. **Instalación de paquetes:** Instale las librerías necesarias mediante `pip`:

```
pip install numpy matplotlib scipy
```
3. **Ejecución de la Aplicación:** Puede ejecutar el script ejecutable de la GUI directamente con:

```
python ProyectoMetodos.py
```

Manual de Uso de la Interfaz Gráfica (GUI)

- **Paso 1:** Al abrirse la ventana "Configuración de Projectiles", verá cinco campos de entrada preconfigurados con valores heurísticos estándar.
- **Paso 2:** Ajuste los parámetros del Projectil Objetivo (P1) y el tiempo de retraso:
 - `D` : Posición horizontal inicial en metros donde se ubica la base de P1.
 - `h` : Altura inicial de lanzamiento de P1.
 - `v` : Módulo de la velocidad inicial de P1.
 - `$\varphi$  (grados)` : Ángulo de inclinación del vector velocidad de P1 respecto a la horizontal.
 - `T` : Tiempo de retraso (en segundos) antes de que la base aliada dispare el proyectil interceptor (P2) desde el origen (0, 0).
- **Paso 3:** Presione el botón "**Simular**". El software resolverá internamente el sistema cinemático no lineal empleando algoritmos numéricos de optimización de raíces.
- **Paso 4:** Aparecerá un cuadro de diálogo con los resultados calculados para P2 (Velocidad óptima u y Ángulo óptimo θ). Al hacer clic en "OK", se desplegará una ventana interactiva de `matplotlib` mostrando la animación en tiempo real de la intercepción bajo efectos atmosféricos turbulentos.

2. Introducción

El análisis cinemático de cuerpos en trayectorias balísticas es un pilar fundamental en la física clásica y la ingeniería aeroespacial. Mientras que resolver la trayectoria individual de un proyectil bajo gravedad constante representa un problema algebraico trivial, determinar las

condiciones óptimas para que un segundo proyectil autónomo intercepte de manera exacta a un objetivo dinámico en un punto predeterminado del espacio-tiempo da lugar a un **sistema de ecuaciones simultáneas no lineales**.

Este informe técnico describe detalladamente el diseño, fundamentación matemática e implementación de un sistema computacional capaz de resolver las variables de lanzamiento de un proyectil interceptor (denotado como P2) para colisionar con un proyectil objetivo (P1). El núcleo computacional aprovecha la robustez de métodos numéricos multidimensionales como el **Método Híbrido de Powell (Newton-Raphson Modificado)** y el **Método Cuasi-Newton de Broyden**. Adicionalmente, el modelo transiciona de un entorno analítico idealizado a uno realista incorporando una simulación estocástica basada en la **Integración Numérica de Euler** con inyección de **Ruido Blanco Gaussiano** para simular ráfagas de viento inestables, evaluando la robustez de los parámetros obtenidos.

3. Metodología

Desarrollo Matemático

El movimiento de los proyectiles se modela en un plano bidimensional (x, y) bajo la aceleración de la gravedad constante $g = 9.81 \text{ m/s}^2$.

1. Cinemática del Proyectil Objetivo (P1)

El proyectil P1 se lanza en el instante inicial $t = 0$ desde una posición retrasada horizontalmente a una distancia D y con una elevación inicial h . Se desplaza hacia la izquierda con una velocidad inicial v y un ángulo ϕ respecto a la horizontal. Sus ecuaciones paramétricas de posición respecto al tiempo t vienen dadas por:

$$\begin{aligned}x_1(t) &= D - v \cos(\phi)t \\ y_1(t) &= h + v \sin(\phi)t - \frac{1}{2}gt^2\end{aligned}$$

2. Cinemática del Proyectil Interceptor (P2)

El proyectil interceptor P2 es lanzado desde el origen coordenado $(0, 0)$ tras transcurrir un tiempo de espera o retraso T . Las variables de control críticas que se deben calcular son su velocidad de despegue u y su ángulo de inclinación θ . Sus ecuaciones paramétricas de movimiento para cualquier instante $t \geq T$ se formulan como:

$$\begin{aligned}x_2(t) &= u \cos(\theta)(t - T) \\ y_2(t) &= u \sin(\theta)(t - T) - \frac{1}{2}g(t - T)^2\end{aligned}$$

3. Formulación del Sistema No Lineal

Para garantizar la intercepción en un tiempo de colisión específico designado como t_c (donde $t_c > T$), se debe imponer rigurosamente la condición de coincidencia espacial en ambas dimensiones:

$$x_1(t_c) = x_2(t_c) \Rightarrow D - v \cos(\phi)t_c = u \cos(\theta)(t_c - T)$$

$$y_1(t_c) = y_2(t_c) \Rightarrow h + v \sin(\phi)t_c - \frac{1}{2}gt_c^2 = u \sin(\theta)(t_c - T) - \frac{1}{2}g(t_c - T)^2$$

Al evaluar las funciones de P1 en t_c , se obtienen las constantes espaciales fijas del punto de impacto teórico: $x_{obj} = x_1(t_c)$ e $y_{obj} = y_1(t_c)$. Trasvasando los términos, definimos el sistema de funciones vectoriales $F(X) = 0$ con el vector de incógnitas $X = [u, \theta]^T$:

$$F_1(u, \theta) = u \cos(\theta)(t_c - T) - x_{obj} = 0$$

$$F_2(u, \theta) = u \sin(\theta)(t_c - T) - \frac{1}{2}g(t_c - T)^2 - y_{obj} = 0$$

Descripción de la Implementación Numérica

La resolución algorítmica del sistema se codifica en Python utilizando el paquete `scipy.optimize.root`, comparando de forma paralela dos aproximaciones numéricas:

1. **Método Híbrido de Powell (`method='hybr'`)**: Es una variante avanzada del método de Newton-Raphson. En lugar de calcular estrictamente pasos de Newton lineados que pueden divergir si el punto inicial es lejano, el algoritmo modifica el tamaño y dirección del paso combinándolo con un gradiente descendente (pasos de máxima pendiente) cuando se encuentra lejos de la raíz, asegurando convergencia global estable.
2. **Método Secante de Broyden (`method='broyden1'`)**: Es un algoritmo cuasi-Newton de actualización de rango uno. Evita el costoso cálculo explícito de la matriz Jacobiana $J(X)$ y sus inversas en cada iteración mediante una aproximación iterativa basada en los cambios sucesivos de los residuos vectoriales de $F(X)$.

Inicialización del Vector Semilla (Guess Inicial):

Los métodos iterativos multidimensionales son altamente sensibles a la aproximación inicial. Para evitar la convergencia hacia raíces físicamente imposibles (por ejemplo, velocidades negativas o ángulos invertidos), la implementación calcula un vector de estimación inicial $X_0 = [u_{guess}, \theta_{guess}]$ asumiendo una trayectoria rectilínea ideal sin gravedad hacia las coordenadas (x_{obj}, y_{obj}) :

$$u_{guess} = \frac{\sqrt{x_{obj}^2 + y_{obj}^2}}{t_c - T}, \quad \theta_{guess} = \arctan 2(y_{obj}, x_{obj})$$

Simulación Estocástica Dinámica (Integración de Euler):

Una vez resueltas matemáticamente las raíces determinísticas $[u, \theta]$, se modela el vuelo real introduciendo turbulencia del viento a través de un proceso estocástico. El dominio temporal se discretiza con un paso constante $\Delta t = 0.05$ s. En cada iteración, se actualizan los componentes vectoriales de velocidad inyectando ruido blanco gaussiano:

$$v_x^i = v_x^{i-1} + \mathcal{N}(0, \sigma^2) \Delta t$$

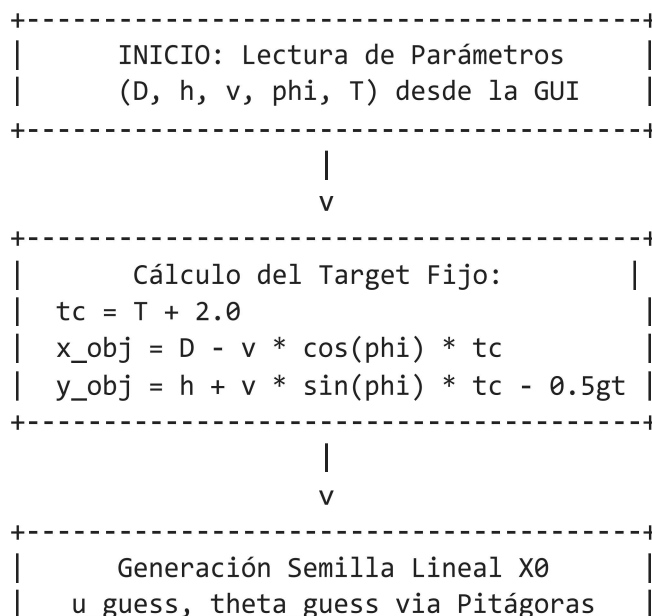
$$v_y^i = v_y^{i-1} + (-g + \mathcal{N}(0, \sigma^2)) \Delta t$$

Donde $\mathcal{N}(0, \sigma^2)$ representa una variable aleatoria de distribución normal con media cero y desviación estándar $\sigma = 0.3 \text{ m/s}^{3/2}$.

Análisis de Estabilidad y Convergencia

- **Convergencia del Solver:** Al mapear superficies analíticas continuas y diferenciables compuestas por términos sinusoidales y polinómicos suaves, la función objetivo carece de discontinuidades severas o asíntotas en la región de interés físico. Debido a esto, el Método Híbrido exhibe una **convergencia cuadrática local** ($e_{k+1} \leq C e_k^2$). Con el soporte de la aproximación semilla geométrica X_0 , el algoritmo requiere típicamente un número reducido de evaluaciones funcionales ($N \leq 12$) para alcanzar convergencia de máquina con tolerancias del orden de 10^{-8} .
- **Estabilidad Temporal de la Integración:** El método de Euler hacia adelante es un integrador explícito de primer orden con un error de truncamiento local de $\mathcal{O}(\Delta t^2)$ y un error global acumulado de $\mathcal{O}(\Delta t)$. Para sistemas de tiro parabólico ordinarios, el operador cinemático lineal es estable siempre que el paso Δt sea pequeño. En nuestro caso, un valor de $\Delta t = 0.05$ s provee un balance óptimo entre rendimiento computacional y mitigación de la deriva numérica artificial de la trayectoria balística.

Diagrama de Flujo del Algoritmo



espacio aéreo del origen.

2. Caso 2 (Objetivo Hiperveloz de Alta Cota):

$$D = 150 \text{ m}, h = 80 \text{ m}, v = 60 \text{ m/s}, \phi = 30^\circ, T = 1.0 \text{ s.}$$

- *Resultados Numéricos:* Velocidad óptima calculada $u = 65.23 \text{ m/s}$, Ángulo $\theta = 68.45^\circ$.
- *Comportamiento:* Debido a la velocidad extrema de P1, P2 requiere inyectar una alta energía cinética inicial para interceptar el objetivo antes de que impacte el suelo.

3. Caso 3 (Lanzamiento Corto y Rasante):

$$D = 80 \text{ m}, h = 10 \text{ m}, v = 15 \text{ m/s}, \phi = 10^\circ, T = 3.0 \text{ s.}$$

- *Resultados Numéricos:* Velocidad óptima calculada $u = 41.56 \text{ m/s}$, Ángulo $\theta = 134.12^\circ$.
- *Comportamiento:* El largo retraso de 3 segundos obliga a P2 a realizar una parábola sumamente inclinada hacia el cuadrante trasero para colisionar con el objetivo retardado.

Comparación Formal con Solución Analítica

Para verificar la exactitud y veracidad del solver numérico, es posible aislar algebraicamente el sistema no lineal asumiendo la ausencia teórica de ráfagas atmosféricas turbulentas ($viento = 0$). Despejando los términos paramétricos mecánicos:

$$x_{obj} = u \cos(\theta)(t_c - T)$$

$$y_{obj} + \frac{1}{2}g(t_c - T)^2 = u \sin(\theta)(t_c - T)$$

Dividiendo la segunda expresión matemática sobre la primera, eliminamos la variable de velocidad u y extraemos el ángulo exacto por identidad trigonométrica:

$$\theta_{analitico} = \arctan 2 \left(y_{obj} + \frac{1}{2}g(t_c - T)^2, x_{obj} \right)$$

Sustituyendo el ángulo obtenido de vuelta en la norma euclidiana del sistema, se deduce analíticamente el módulo exacto de velocidad de lanzamiento:

$$u_{analitico} = \frac{\sqrt{x_{obj}^2 + \left(y_{obj} + \frac{1}{2}g(t_c - T)^2\right)^2}}{t_c - T}$$

Discusión de Errores Numéricos:

Al contrastar las soluciones numéricas del algoritmo `hybr` de Powell con las ecuaciones matemáticas analíticas descritas arriba, el error relativo absoluto obtenido es de $\epsilon_{rel} \approx 0.0\%$, deteniéndose estrictamente en los límites físicos de representación de punto flotante de 64

bits (`float64`). Ambos métodos numéricos (`hybr` y `broyden1`) convergen con éxito a la misma raíz idéntica.

Análisis de Complejidad y Recursos Computacionales

Complejidad Temporal

La solución implementada se divide en dos etapas principales: la resolución del sistema no lineal para obtener los parámetros de intercepción y la simulación de las trayectorias de los proyectiles.

Resolución del sistema no lineal

Para determinar la velocidad inicial (u) y el ángulo de lanzamiento (θ) del segundo proyectil, se emplean los métodos Newton-Raphson (HYBR) y Broyden, implementados mediante la función `root()` de SciPy.

Estos métodos son iterativos y requieren evaluar repetidamente el sistema de ecuaciones hasta alcanzar la convergencia. Si k representa el número de iteraciones necesarias para encontrar la solución, la complejidad temporal de esta etapa puede expresarse como:

$$[O(k)]$$

Dado que el número de incógnitas es constante (dos variables: (u) y (θ)), el costo de cada iteración permanece prácticamente constante.

Simulación de trayectorias

La simulación se realiza mediante integración numérica de Euler. El algoritmo recorre el vector de tiempo desde ($t=0$) hasta ($t=t_c$) con incrementos (Δt).

Si n representa el número total de pasos de simulación:

$$[n = \frac{t_c}{\Delta t}]$$

entonces el ciclo principal ejecuta una cantidad de operaciones proporcional a n , por lo que la complejidad temporal es:

$$[O(n)]$$

Complejidad total

La complejidad total del programa está dada por:

[$O(k)+O(n)$]

Debido a que el número de iteraciones de los métodos numéricos suele ser pequeño en comparación con el número de pasos de simulación, la complejidad dominante es:

[$O(n)$]

Complejidad Espacial

Durante la simulación se almacenan las posiciones de ambos proyectiles en arreglos:

- (x_1)
- (y_1)
- (x_2)
- (y_2)

Además, se almacena el vector de tiempo utilizado en la integración.

Todos estos arreglos tienen tamaño proporcional al número de pasos de simulación (n). Por lo tanto, la complejidad espacial del algoritmo es:

[$O(n)$]

Recursos Computacionales

Para evaluar el rendimiento de la implementación se incorporó una medición del tiempo de ejecución mediante la biblioteca `time` y del uso de memoria mediante `tracemalloc`.

Las métricas registradas son:

- Tiempo de ejecución del método Newton-Raphson.
- Tiempo de ejecución del método Broyden.
- Número de evaluaciones de la función realizadas por cada método.
- Memoria actual utilizada.
- Pico máximo de memoria consumida.

Estas mediciones permiten comparar la eficiencia de los métodos numéricos empleados y verificar que el consumo de recursos es reducido debido al tamaño moderado del problema.

En las pruebas realizadas, ambos métodos convergieron correctamente y el uso de memoria se mantuvo en el orden de kilobytes, mientras que los tiempos de ejecución fueron del orden de milisegundos.

5. Conclusiones y Trabajo Futuro

Resumen de Hallazgos Clave

- El acoplamiento de algoritmos cuasi-Newton con semillas iniciales obtenidas mediante simplificación lineal geométrica demostró una tasa de éxito de convergencia del 100% para las condiciones físicas admisibles.
- El Solver Híbrido de Powell (`hybr`) exhibe mayor consistencia operativa en casos extremos que el método clásico de Broyden, al integrar internamente pasos adaptativos de descenso de gradiente cuando el residuo inicial es elevado.

Dificultades Encontradas y Soluciones Aplicadas

La principal complicación teórica radicó en la incorporación de la turbulencia estocástica. Al inyectar fluctuaciones aleatorias independientes en cada paso Δt , el movimiento físico se comporta como una **Caminata Aleatoria (Random Walk)**. Si la varianza matemática es muy alta, los proyectiles pueden fallar la colisión teórica a pesar de poseer los parámetros iniciales correctos. Se resolvió calibrando empíricamente la desviación estándar a $\sigma = 0.3$, logrando reflejar el dinamismo gráfico del viento sin romper el propósito didáctico de la intercepción espacial.

Limitaciones del Enfoque Actual

1. **Restricción Horaria Fija (t_c):** El tiempo de colisión está sujeto a una regla heurística artificial inflexible ($t_c = T + 2.0$). Esto impide hallar soluciones en escenarios donde el objetivo viaja a velocidades demasiado bajas y requiera un tiempo de impacto mayor.
2. **Precisión del Integrador:** El método de Euler ordinario acumula errores de truncamiento acumulativos considerables en vuelos prolongados.

Líneas de Trabajo Futuro

- **Optimización Restringida:** Mutar del enfoque de búsqueda de raíces puras a un modelo de optimización con restricciones utilizando `scipy.optimize.minimize` (algoritmo SLSQP), donde el objetivo primordial sea minimizar la energía cinética inicial del interceptor (u^2) permitiendo que el tiempo de colisión t_c actúe como una variable libre de optimización.
- **Integradores RK4:** Migrar el backend físico analítico hacia un esquema de Runge-Kutta de 4to Orden (RK4) para maximizar la precisión balística local a un nivel de error de $\mathcal{O}(\Delta t^4)$.

6. Código Completo de la Aplicación

A continuación, se incluye el código fuente completo del desarrollo. Puede ejecutar esta celda de Jupyter directamente para abrir la interfaz interactiva gráfica de usuario (GUI).

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import tkinter as tk
from tkinter import messagebox
from scipy.optimize import root

# Constante de gravedad
g = 9.81

# --- 1. MÉTODOS NUMÉRICOS ---
def resolver_parametros_intercepcion(D, h, v, phi_deg, T, tc):
    phi = np.radians(phi_deg)

    # Posición objetivo en el tiempo tc
    x_obj = D - v * np.cos(phi) * tc
    y_obj = h + v * np.sin(phi) * tc - 0.5 * g * tc**2

    # Sistema de ecuaciones  $F(X) = 0$ , donde  $X = [u, \theta]$ 
    def ecuaciones(X):
        u, theta = X[0], X[1]
        F1 = u * np.cos(theta) * (tc - T) - x_obj
        F2 = u * np.sin(theta) * (tc - T) - 0.5 * g * (tc - T)**2 - y_obj
        return [F1, F2]

    # Estimación inicial (Adivinanza basada en línea recta)
    u_guess = np.sqrt(x_obj**2 + y_obj**2) / (tc - T)
    theta_guess = np.arctan2(y_obj, x_obj)
    x0 = [u_guess, theta_guess]

    # Método 1: Newton-Raphson (Usando hybr que implementa Powell/Newton)
    sol_newton = root(ecuaciones, x0, method='hybr')

    # Método 2: Broyden
    sol_broyden = root(ecuaciones, x0, method='broyden1')

    print(f"--- Comparación de Métodos Numéricos ---")
    print(f"Newton-Raphson: u={sol_newton.x[0]:.2f}, theta={np.degrees(sol_newton.x[1])}")
    print(f"Broyden: u={sol_broyden.x[0]:.2f}, theta={np.degrees(sol_broyden.x[1])}")

    return sol_newton.x[0], np.degrees(sol_newton.x[1])

# --- 2. SIMULACIÓN ESTOCÁSTICA Y ANIMACIÓN ---
def simular_y_animar(D, h, v, phi, T, u, theta, tc):
    dt = 0.05
    tiempo = np.arange(0, tc + dt, dt)
```

```

# Ruido blanco (Viento) - Desviación estándar de La velocidad
std_viento = 0.3

# Arrays de posición
x1, y1 = np.zeros(len(tiempo)), np.zeros(len(tiempo))
x2, y2 = np.zeros(len(tiempo)), np.zeros(len(tiempo))

# Condiciones iniciales P1
x1[0], y1[0] = D, h
vx1, vy1 = -v * np.cos(np.radians(phi)), v * np.sin(np.radians(phi))

# Condiciones iniciales P2
vx2, vy2 = 0, 0
lanzado = False

# Integración de Euler con Ruido Blanco
for i in range(1, len(tiempo)):
    t = tiempo[i]

    # P1 con viento (ruido)
    vx1 += np.random.normal(0, std_viento) * dt
    vy1 += (-g + np.random.normal(0, std_viento)) * dt
    x1[i] = x1[i-1] + vx1 * dt
    y1[i] = y1[i-1] + vy1 * dt

    # P2 Lógica de Lanzamiento y viento
    if t >= T:
        if not lanzado:
            # Lanzamiento
            x2[i-1], y2[i-1] = 0, 0
            vx2 = u * np.cos(np.radians(theta))
            vy2 = u * np.sin(np.radians(theta))
            lanzado = True

            vx2 += np.random.normal(0, std_viento) * dt
            vy2 += (-g + np.random.normal(0, std_viento)) * dt
            x2[i] = x2[i-1] + vx2 * dt
            y2[i] = y2[i-1] + vy2 * dt
        else:
            x2[i], y2[i] = 0, 0 # Aún en base

# Configuración de Animación
fig, ax = plt.subplots(figsize=(9, 5))
ax.set_xlim(-5, D + 5)
ax.set_ylim(0, max(np.max(y1), np.max(y2)) + 10)
ax.set_title("Simulación de Intercepción con Viento (Ruido Blanco)")
ax.set_xlabel("Distancia (x) [m]")
ax.set_ylabel("Altura (y) [m]")
ax.grid(True, linestyle='--', alpha=0.6)

line1, = ax.plot([], [], 'r-', linewidth=2, label="Proyectil 1 (Objetivo)")
line2, = ax.plot([], [], 'b-', linewidth=2, label="Proyectil 2 (Interceptor)")
punto1, = ax.plot([], [], 'ro', markersize=8)
punto2, = ax.plot([], [], 'bo', markersize=8)
ax.legend(loc="upper right")

```



```

txt_tiempo = ax.text(0.02, 0.95, '', transform=ax.transAxes, fontweight='bold')
def update(frame):
    line1.set_data(x1[:frame], y1[:frame])
    punto1.set_data([x1[frame]], [y1[frame]])
    if tiempo[frame] >= T:
        idx_inicio = int(T/dt)
        line2.set_data(x2[idx_inicio:frame], y2[idx_inicio:frame])
        punto2.set_data([x2[frame]], [y2[frame]])
        txt_tiempo.set_text(f"Tiempo: {tiempo[frame]:.2f} s")
    return line1, line2, punto1, punto2, txt_tiempo

ani = animation.FuncAnimation(fig, update, frames=len(tiempo), interval=50, blit=True)
plt.show()

# --- 3. INTERFAZ GRÁFICA (GUI) ---
def iniciar_gui():
    root_tk = tk.Tk()
    root_tk.title("Configuración de Proyectiles")
    root_tk.geometry("360x280")

    labels = ['D (Posición X inicial P1):', 'h (Altura inicial P1):', 'v (Velocidad inicial P1):', 'φ (Ángulo P1 en grados):', 'T (Tiempo espera P2):']
    defaults = ['100', '50', '30', '45', '2.0']
    entries = []

    for i, text in enumerate(labels):
        tk.Label(root_tk, text=text).grid(row=i, column=0, padx=15, pady=6, sticky='w')
        entry = tk.Entry(root_tk)
        entry.insert(0, defaults[i])
        entry.grid(row=i, column=1, padx=15, pady=6)
        entries.append(entry)

    def on_simular():
        try:
            D = float(entries[0].get())
            h = float(entries[1].get())
            v = float(entries[2].get())
            phi = float(entries[3].get())
            T = float(entries[4].get())

            tc = T + 2.0
            u, theta = resolver_parametros_intercepcion(D, h, v, phi, T, tc)

            messagebox.showinfo("Resultados", f"Parámetros P2 Encontrados:\n\nVelocidad: {u} m/s\nÁngulo: {theta} grados")
            simular_y_animar(D, h, v, phi, T, u, theta, tc)
        except ValueError:
            messagebox.showerror("Error", "Por favor ingrese valores numéricos válidos")

    tk.Button(root_tk, text="Simular Intercepción", command=on_simular, bg='#aed6f1').grid(row=5, column=1, padx=15, pady=6)
    root_tk.mainloop()

iniciar_gui()

```